Nama : Oktova Yoga Praditia Samosir

NIM : 42324001

Prodi : D3 - Teknologi Informasi

Soal 1

1. Load and preview the data

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

path = "users_data_cleaned.pickle"
df = pd.read_pickle(path)
df.head()
```

Out[ ]:

	user_id	number_transactions	total_amount_usd	age	job	marital	educ
0	9231c446-cb16-4b2b-a7f7-ddfc8b25aaf6	3.0	2143.0	58	management	married	te
1	bb92765a-08de-4963-b432-496524b39157	NaN	NaN	44	technician	single	seco
2	573de577-49ef-42b9-83da-d3cfb817b5c1	2.0	2.0	33	entrepreneur	married	seco
3	d6b66b9d-7c8f-4257-a682-e136f640b7e3	NaN	NaN	47	blue-collar	married	
4	fade0b20-7594-4d9a-84cd-c02f79b1b526	1.0	1.0	33	None	single	

2. Check the shape of the data

In [ ]:
```
print("Jumlah baris dan kolom:", df.shape)
```

```
Jumlah baris dan kolom: (45209, 19)
```

3. Check the data types

In [ ]:
```
df.dtypes
```

Out[ ]:
```
user_id                       object
number_transactions          float64
total_amount_usd             float64
age                            int64
job                           object
marital                       object
education                     object
default                         bool
housing                         bool
loan                            bool
contact                       object
duration                       int64
campaign                       int64
pdays                          int64
previous                       int64
poutcome                      object
term_deposit                    bool
date_joined          datetime64[ns]
device                        object
dtype: object
```

4. Check for correlations

In [ ]:
```
df.corr(numeric_only=True)
```

Out[ ]:

	number_transactions	total_amount_usd	age	default	housin
number_transactions	1.000000	-0.163409	-0.008813	0.138838	0.03042
total_amount_usd	-0.163409	1.000000	0.095839	-0.065390	-0.06685
age	-0.008813	0.095839	1.000000	-0.017875	-0.18555
default	0.138838	-0.065390	-0.017875	1.000000	-0.00602
housing	0.030429	-0.066857	-0.185552	-0.006020	1.00000
loan	0.075319	-0.084526	-0.015641	0.077232	0.04134
duration	-0.017220	0.022586	-0.004645	-0.010017	0.00504
campaign	0.026431	-0.017274	0.004767	0.016819	-0.02358
pdays	-0.030751	0.006435	-0.023745	-0.029982	0.12419
previous	-0.023046	0.016952	0.001297	-0.018331	0.03708
term_deposit	-0.053390	0.050785	0.025168	-0.022421	-0.13916

5. Generate summary statistics for numerical data only

In [ ]:
```
df.select_dtypes(include='number').describe()
```

Out[ ]:

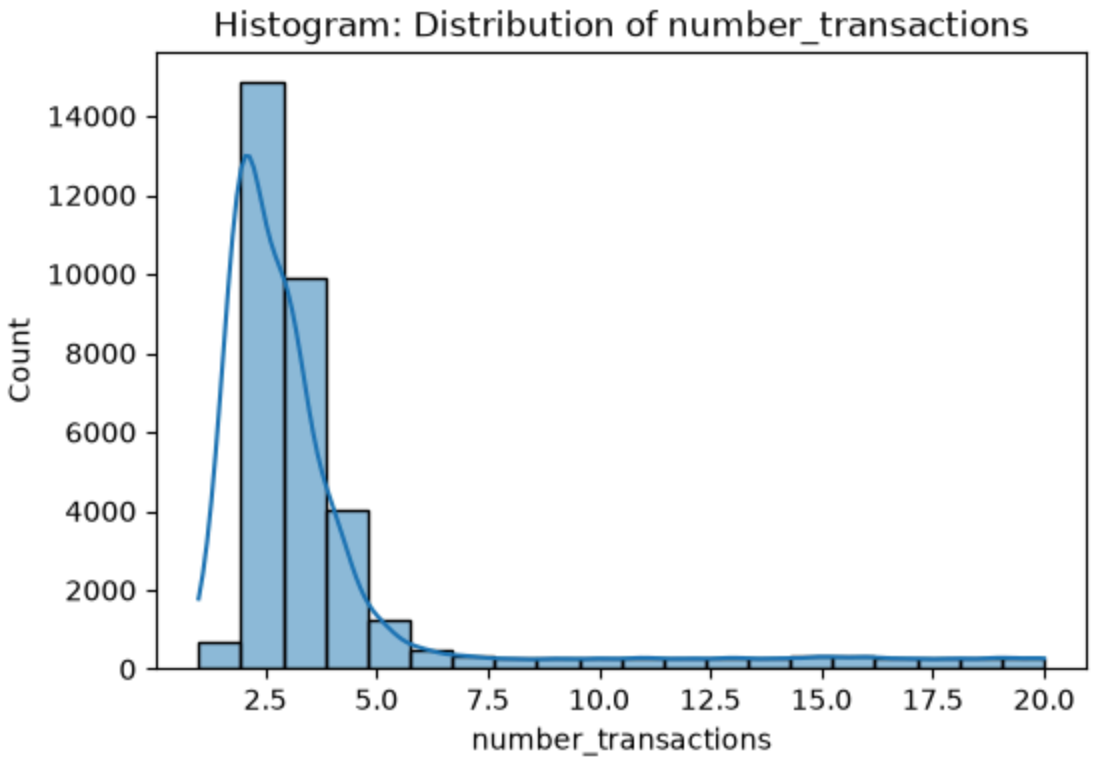
	number_transactions	total_amount_usd	age	duration	campaign
count	35210.000000	35210.000000	45209.000000	45209.000000	45209.000000
mean	3.977052	1369.417751	40.935853	258.153067	2.763897
std	3.814329	3063.412688	10.618653	257.525446	3.098076
min	1.000000	-8019.000000	18.000000	0.000000	1.000000
25%	2.000000	73.000000	33.000000	103.000000	1.000000
50%	3.000000	451.000000	39.000000	180.000000	2.000000
75%	4.000000	1438.000000	48.000000	319.000000	3.000000
max	20.000000	102127.000000	95.000000	4918.000000	63.000000

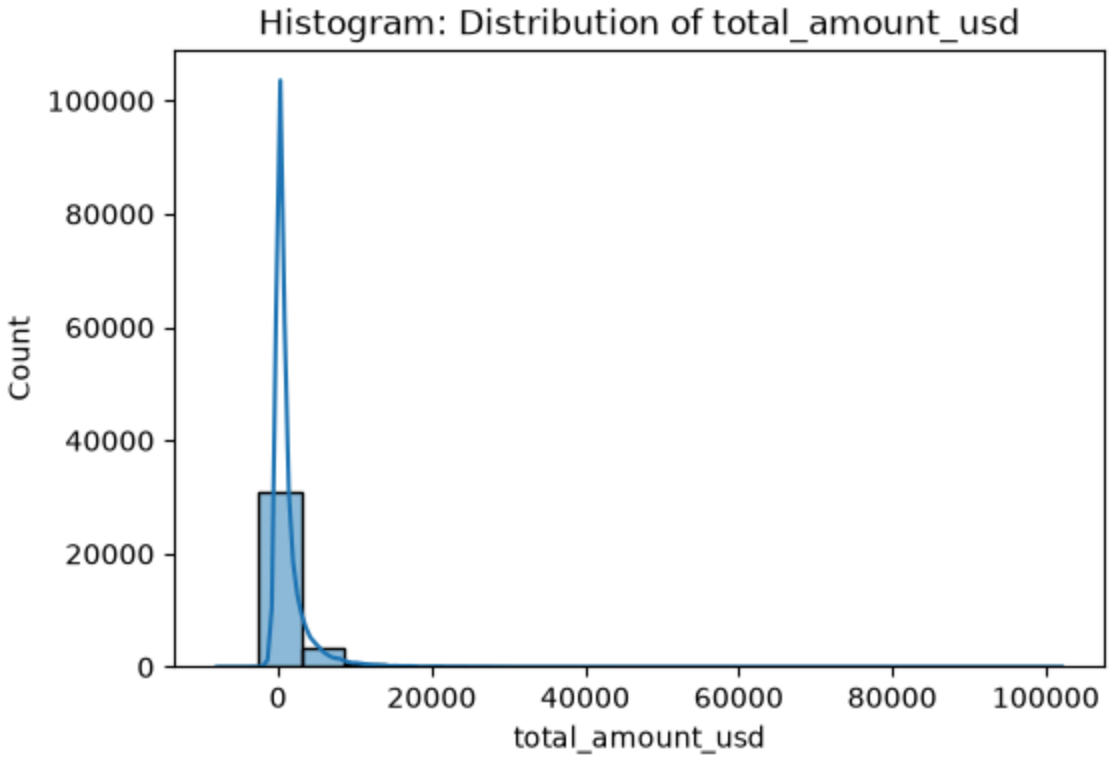
6. Plot Histograms for all numerical columns

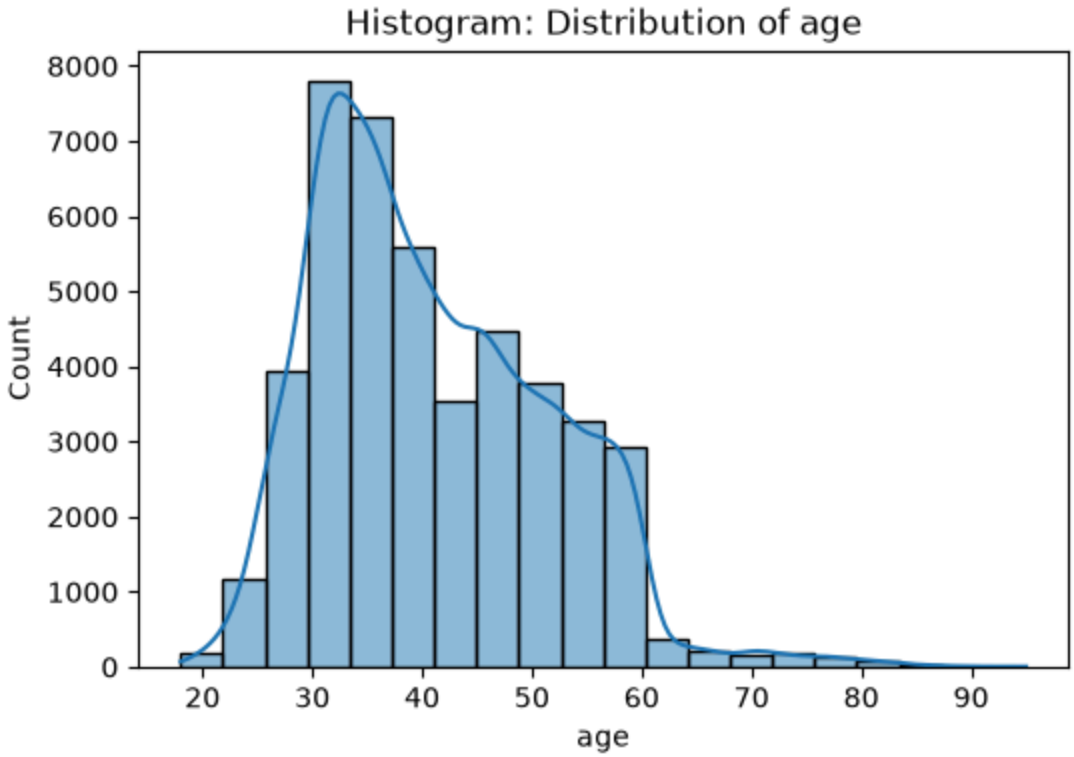
In [ ]:
```
numeric_cols = df.select_dtypes(include='number').columns

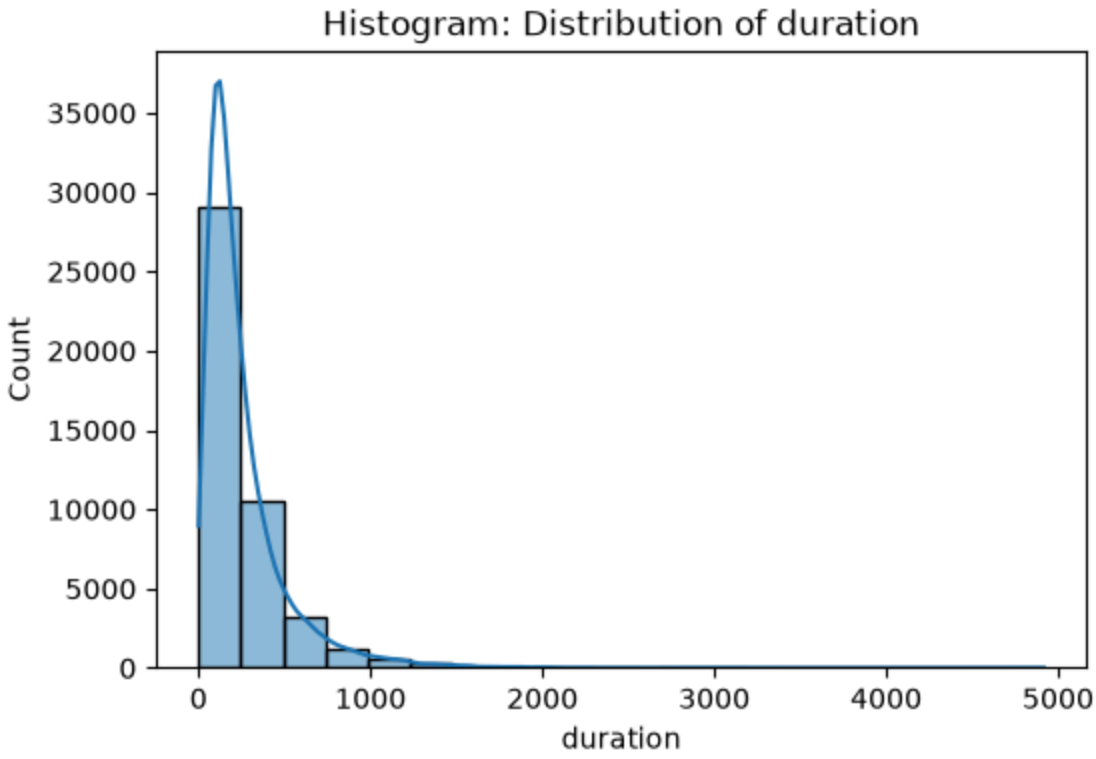
for col in numeric_cols:
    plt.figure(figsize=(6,4))
    sns.histplot(df[col], bins=20, kde=True)
```

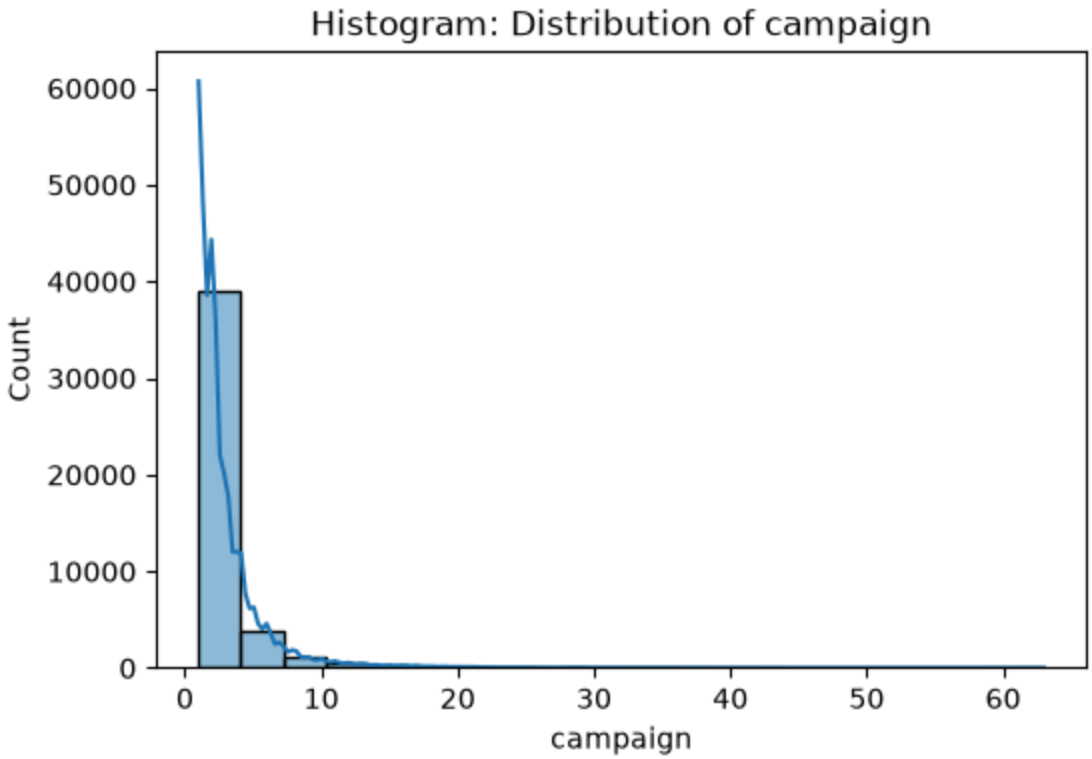
```
    plt.title(f"Histogram: Distribution of {col}")
    plt.show()
```

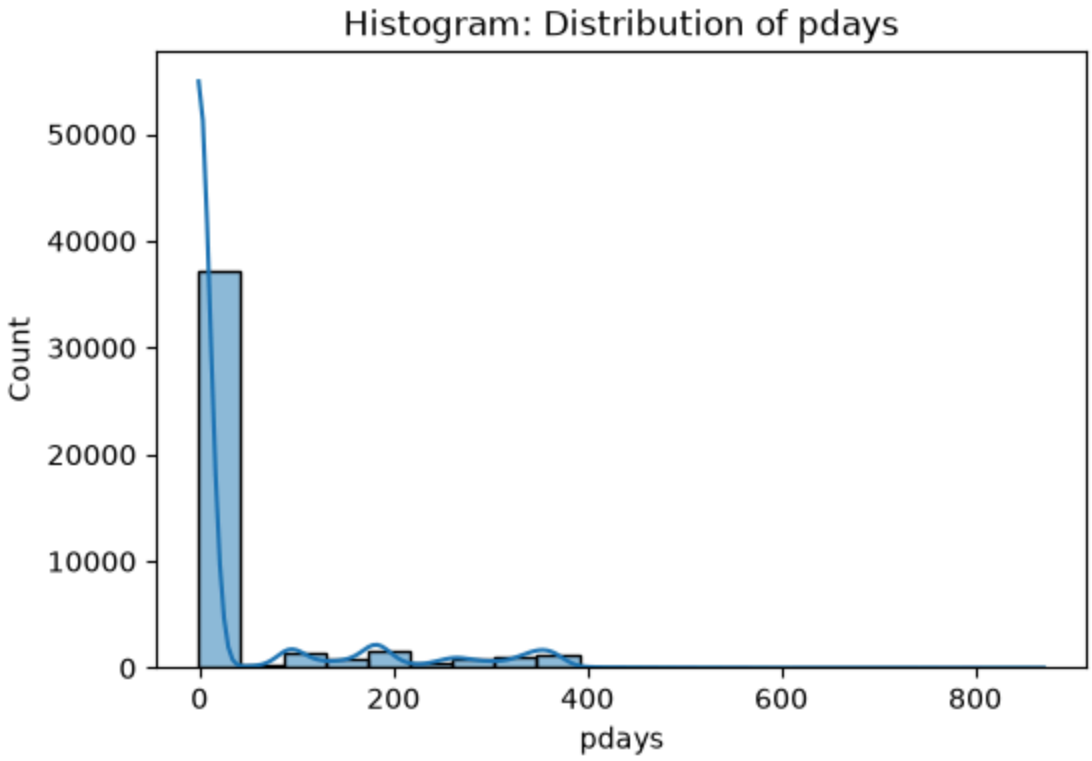


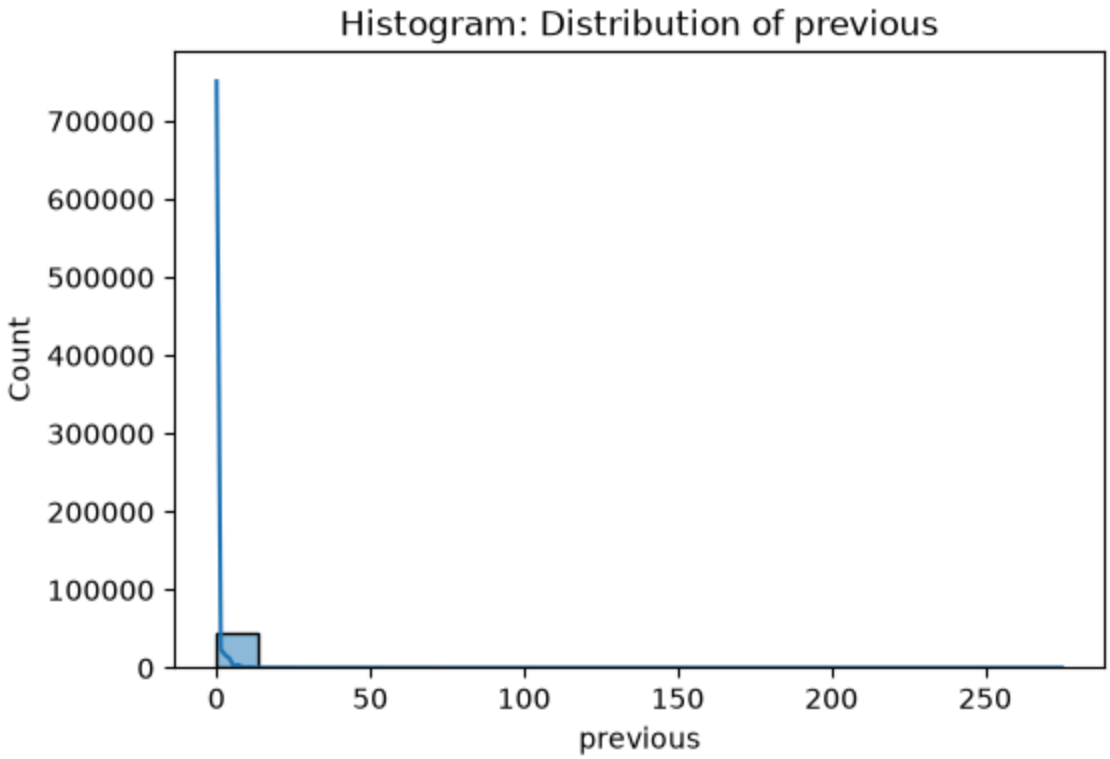



Histogram: Distribution of age
Count
8000
7000
6000
5000
4000
3000
2000
1000
0
20
30
40
50
60
70
80
90
age


Histogram: Distribution of duration
Count
35000
30000
25000
20000
15000
10000
5000
0
0
1000
2000
3000
4000
5000
duration


Histogram: Distribution of campaign
Count
campaign
0
10000
20000
30000
40000
50000
60000
0
10
20
30
40
50
60

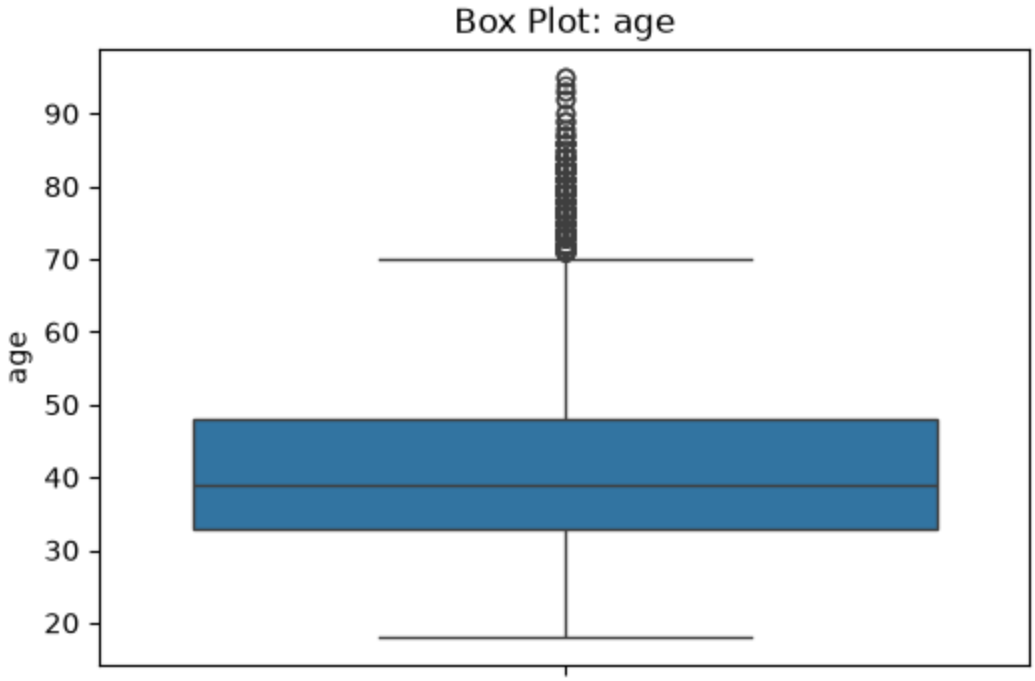

Histogram: Distribution of pdays
Count
pdays
0
10000
20000
30000
40000
50000
0
200
400
600
800

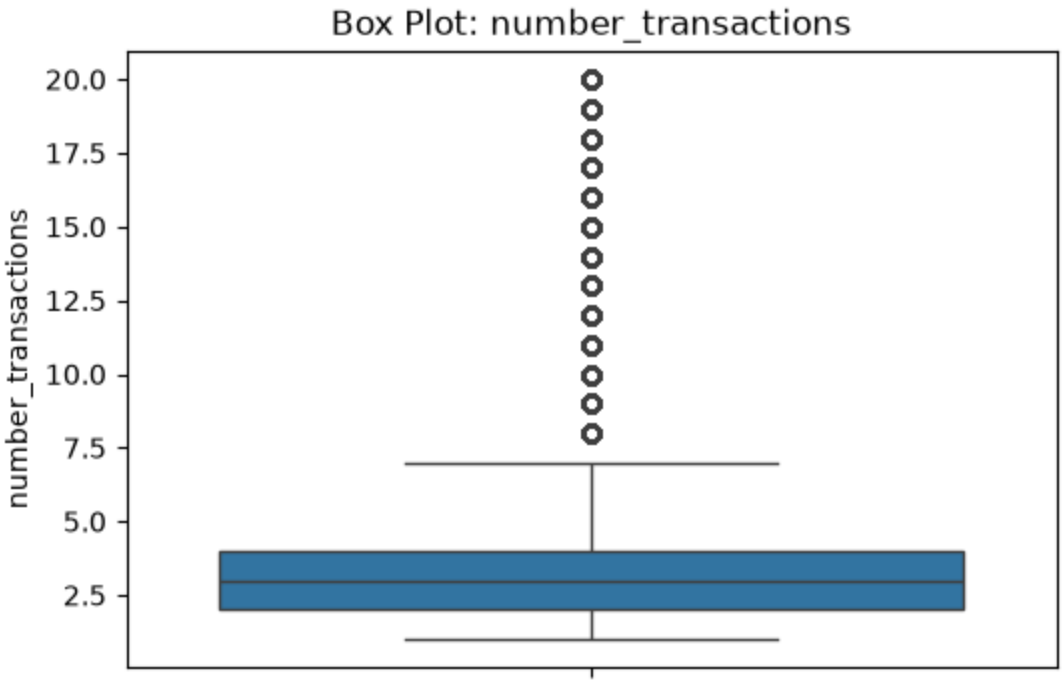


7. Generate a box plot for age and number_transactions

In [ ]:
```
plt.figure(figsize=(6,4))
sns.boxplot(y=df['age'])
plt.title("Box Plot: age")
plt.show()

plt.figure(figsize=(6,4))
sns.boxplot(y=df['number_transactions'])
plt.title("Box Plot: number_transactions")
plt.show()
```

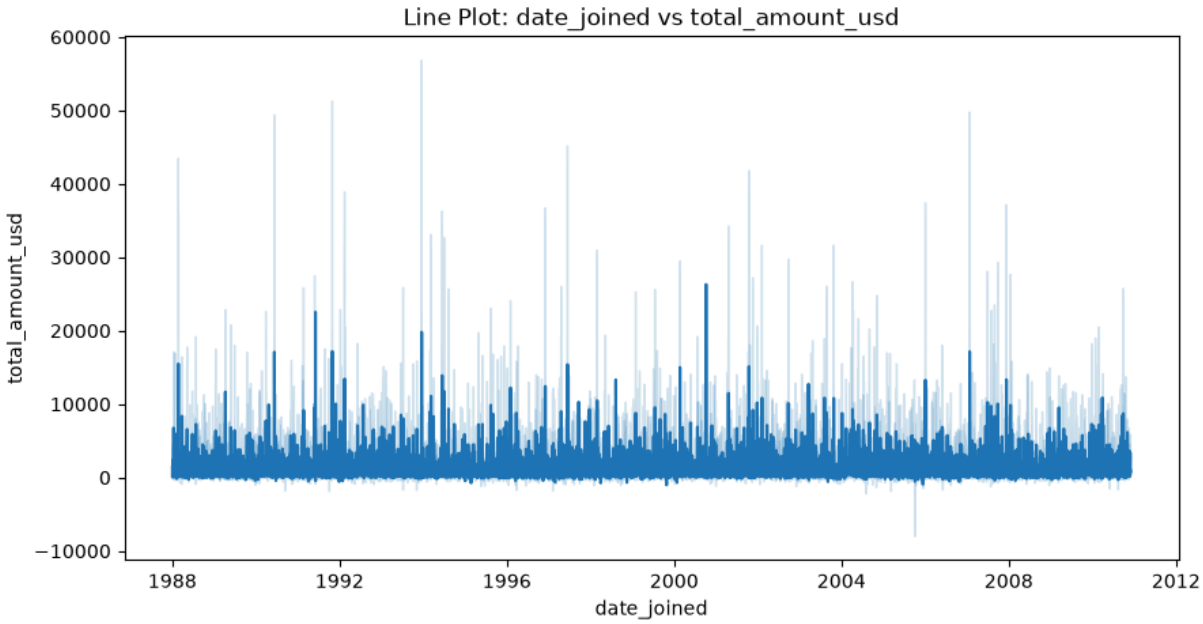




8. Generate a lineplot for date_joined to total_amount_usd

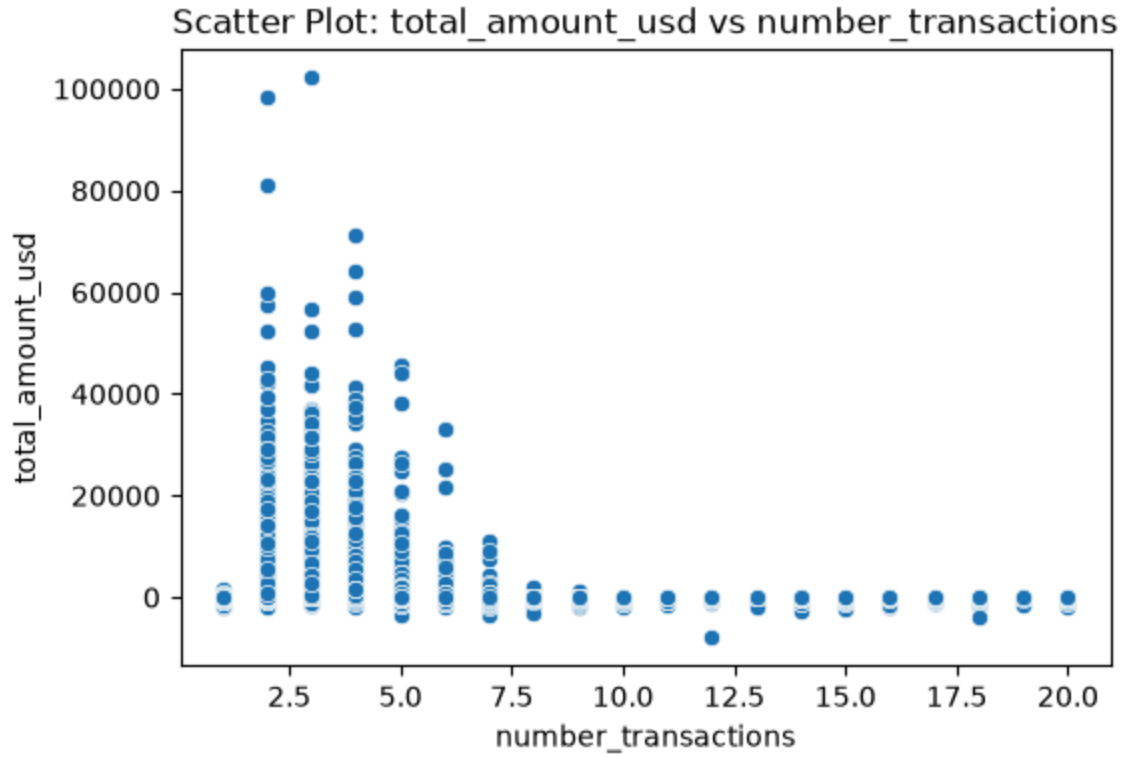
In [ ]:
```
df_sorted = df.sort_values('date_joined')

plt.figure(figsize=(10,5))
sns.lineplot(x='date_joined', y='total_amount_usd', data=df_sorted)
plt.title("Line Plot: date_joined vs total_amount_usd")
plt.xlabel("date_joined")
plt.ylabel("total_amount_usd")
plt.show()
```



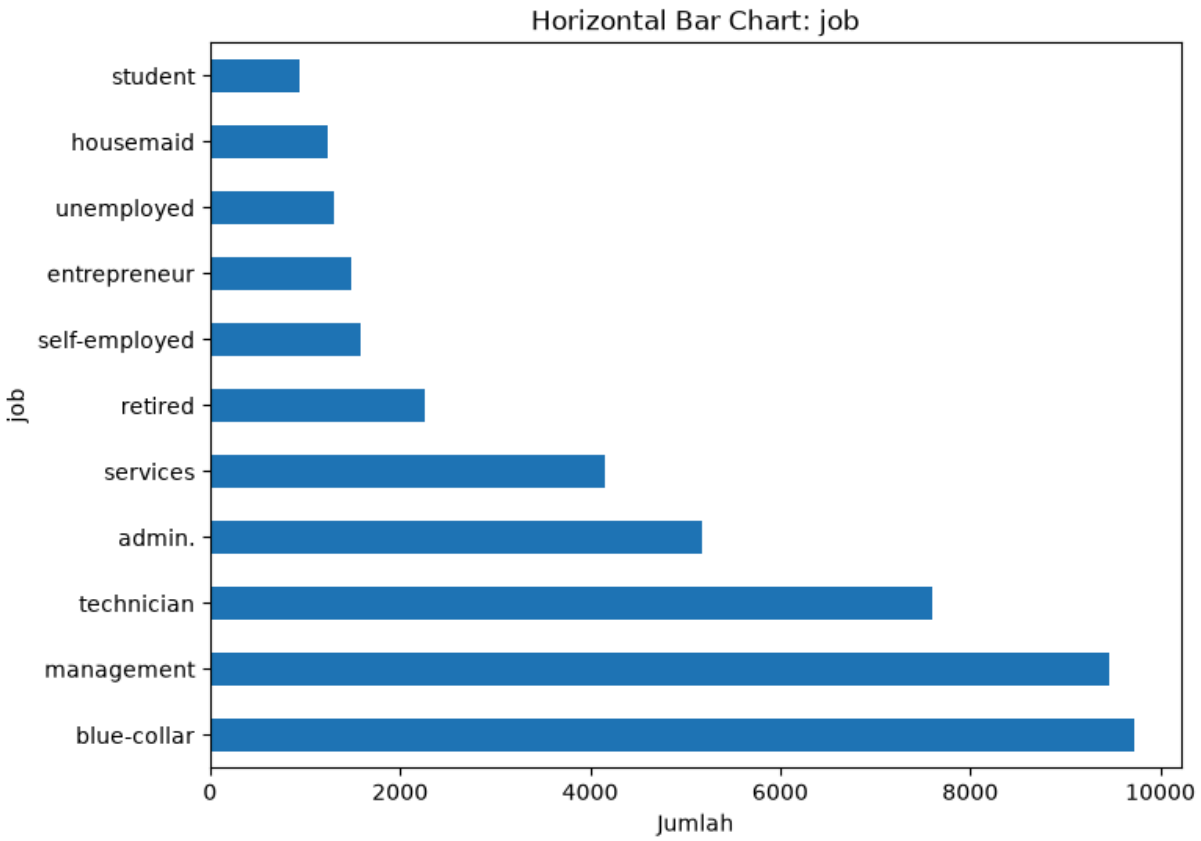
9. Generate scatter plots comparing total_amount_usd to number_transaction

In [ ]:
```
plt.figure(figsize=(6,4))
sns.scatterplot(x='number_transactions', y='total_amount_usd', data=df)
plt.title("Scatter Plot: total_amount_usd vs number_transactions")
plt.show()
```



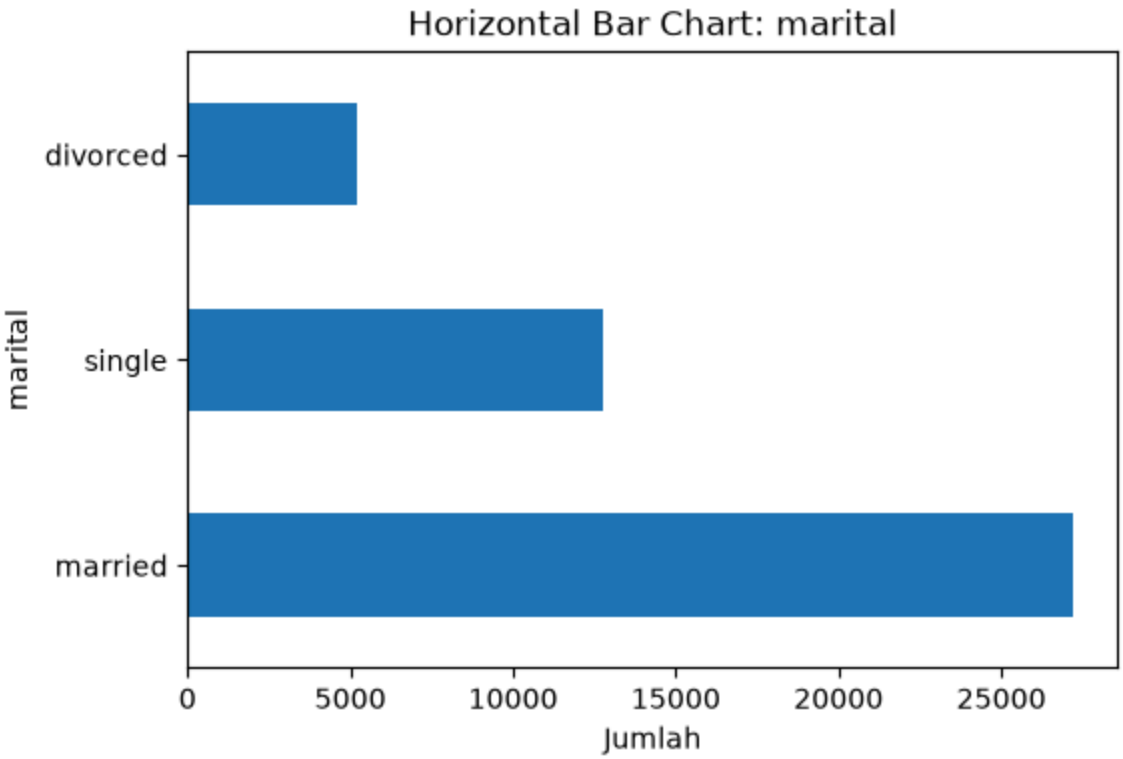
10. Generate Horizontal bar charts for job

In [ ]:
```
plt.figure(figsize=(8,6))
df['job'].value_counts().plot(kind='barh')
plt.title("Horizontal Bar Chart: job")
plt.xlabel("Jumlah")
plt.ylabel("job")
plt.show()
```



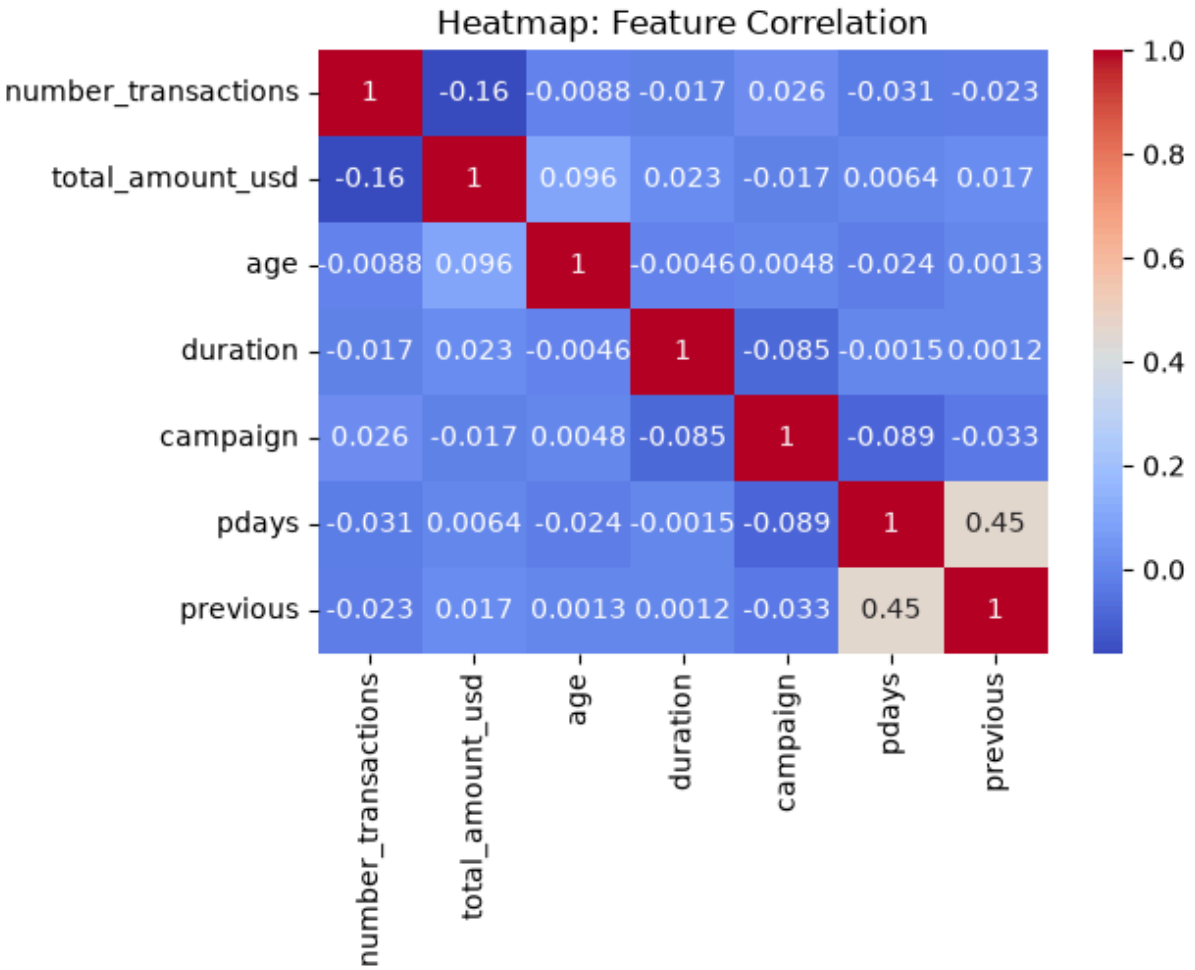
11. Generate Horizontal bar charts for marital

In [ ]:
```
plt.figure(figsize=(6,4))
df['marital'].value_counts().plot(kind='barh')
plt.title("Horizontal Bar Chart: marital")
plt.xlabel("Jumlah")
plt.ylabel("marital")
plt.show()
```



12. Generate a heatmap for feature correlations

In [ ]:
```
plt.figure(figsize=(6,4))
sns.heatmap(df.select_dtypes(include='number').corr(), annot=True, cmap='coolwarm')
plt.title("Heatmap: Feature Correlation")
plt.show()
```



13. Identify Missing Values

In [ ]:
```
df.isnull().sum()
```

Out[ ]:
```
user_id                 0
number_transactions     9999
total_amount_usd        9999
age                     0
job                     288
marital                 0
education               1857
default                 0
housing                 0
loan                    0
contact                 13018
duration                0
campaign                0
pdays                   0
previous                0
poutcome                36957
term_deposit            0
date_joined             30
device                  94
dtype: int64
```

Penjelasan:

df.isnull().sum() memeriksa dan menjumlahkan seluruh data kosong "missing value" pada masing-masing kolom. Output ini memberikan gambaran langsung kolom mana saja yang memiliki nilai kosong dalam dataset.

In [ ]:
```
percent_missing = 100*df.isnull().mean()
percent_missing
```

Out[ ]:
```
user_id                 0.000000
number_transactions    22.117278
total_amount_usd       22.117278
age                     0.000000
job                     0.637041
marital                 0.000000
education               4.107589
default                 0.000000
housing                 0.000000
loan                    0.000000
contact                28.795151
duration                0.000000
campaign                0.000000
pdays                   0.000000
previous                0.000000
poutcome               81.746997
term_deposit            0.000000
date_joined             0.066358
device                  0.207923
dtype: float64
```

Penjelasan:

Menghitung persentase nilai kosong untuk setiap kolom dengan membagi jumlah missing value terhadap total baris dan dikalikan 100%. Informasi persentase ini penting untuk menentukan apakah data kosong tersebut perlu diimputasi (diisi) atau dihapus jika persentasenya terlalu besar.

Soal 2

1. Membaca file real_estate.csv

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('real_estate.csv')
print(df.head())
```

```
   No  X1 transaction date  X2 house age  \
0   1             2012.917          32.0
1   2             2012.917          19.5
2   3             2013.583          13.3
3   4             2013.500          13.3
4   5             2012.833           5.0

   X3 distance to the nearest MRT station  X4 number of convenience stores  \
0                                84.87882                                10
1                               306.59470                                 9
2                               561.98450                                 5
3                               561.98450                                 5
4                               390.56840                                 5

   X5 latitude  X6 longitude  Y house price of unit area
0     24.98298     121.54024                        37.9
1     24.98034     121.53951                        42.2
2     24.98746     121.54391                        47.3
3     24.98746     121.54391                        54.8
4     24.97937     121.54245                        43.1
```

2. Menampilkan variable dalam dataset

In [ ]:
```
print("Nama kolom:", list(df.columns))


X = df.iloc[:, 1:-1].values
y = df.iloc[:, -1].values

X1 = df['X1 transaction date'].values
X2 = df['X2 house age'].values
X3 = df['X3 distance to the nearest MRT station'].values
X4 = df['X4 number of convenience stores'].values
X5 = df['X5 latitude'].values
X6 = df['X6 longitude'].values
Y = df['Y house price of unit area'].values

print("Variabel X1 (transaction date), 5 data pertama:\n", X1[:5])
print("Variabel X2 (house age), 5 data pertama:\n", X2[:5])
print("Variabel X3 (distance to the nearest MRT station), 5 data pertama:\n", X3[:5
print("Variabel X4 (number of convenience stores), 5 data pertama:\n", X4[:5])
print("Variabel X5 (latitude), 5 data pertama:\n", X5[:5])
print("Variabel X6 (longitude), 5 data pertama:\n", X6[:5])
print("Variabel Y (house price of unit area), 5 data pertama:\n", Y[:5])
print("\nDimensi X:", X.shape)
print("Dimensi y:", y.shape)
```

```
Nama kolom: ['No', 'X1 transaction date', 'X2 house age', 'X3 distance to the neares
t MRT station', 'X4 number of convenience stores', 'X5 latitude', 'X6 longitude', 'Y
house price of unit area']
Variabel X1 (transaction date), 5 data pertama:
 [2012.917 2012.917 2013.583 2013.5   2012.833]
Variabel X2 (house age), 5 data pertama:
 [32.  19.5 13.3 13.3  5. ]
Variabel X3 (distance to the nearest MRT station), 5 data pertama:
 [ 84.87882 306.5947  561.9845  561.9845  390.5684 ]
Variabel X4 (number of convenience stores), 5 data pertama:
 [10  9  5  5  5]
Variabel X5 (latitude), 5 data pertama:
 [24.98298 24.98034 24.98746 24.98746 24.97937]
Variabel X6 (longitude), 5 data pertama:
 [121.54024 121.53951 121.54391 121.54391 121.54245]
Variabel Y (house price of unit area), 5 data pertama:
 [37.9 42.2 47.3 54.8 43.1]

Dimensi X: (414, 6)
Dimensi y: (414,)
```

3. Menampilkan data yang di-split train: 80% & test: 20%

In [ ]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

print("X_train:\n", X_train)
print("y_train:\n", y_train)
print("X_test:\n", X_test)
print("y_test:\n", y_test)
print("Jumlah data train:", len(X_train), "| Jumlah data test:", len(X_test))
```

```
X_train:
 [[2.0135000e+03 1.6500000e+01 2.2880110e+03 3.0000000e+00 2.4958850e+01
  1.2151359e+02]
 [2.0134170e+03 4.5000000e+00 2.2758770e+03 3.0000000e+00 2.4963140e+01
  1.2151151e+02]
 [2.0135000e+03 3.8300000e+01 4.3971050e+02 0.0000000e+00 2.4971610e+01
  1.2153423e+02]
 ...
 [2.0130000e+03 1.3600000e+01 4.1973490e+03 0.0000000e+00 2.4938850e+01
  1.2150383e+02]
 [2.0135830e+03 3.5900000e+01 6.4073910e+02 3.0000000e+00 2.4975630e+01
  1.2153715e+02]
 [2.0135830e+03 6.6000000e+00 9.0456060e+01 9.0000000e+00 2.4974330e+01
  1.2154310e+02]]
y_train:
 [ 23.2  29.3  38.4  37.5  34.3  44.8  54.4  37.9   7.6  29.5  25.3  13.7
  30.   30.8  40.6  20.7  40.8  55.3  40.5  40.8  40.3  33.1  26.2  41.9
  18.8  78.   15.6  39.4  41.   23.5  35.6  47.1  35.6  55.   41.5  37.4
  21.8  31.3  36.8  26.6  40.   50.   38.1  59.5  56.8  25.3  32.4  37.8
  49.8  52.7  21.8  28.8  29.3  37.4  37.4  44.   50.7  42.3  36.5  51.4
  32.9  38.6  21.4  50.4  37.4  52.2  58.8  43.8  22.1  53.3  17.4  45.7
  36.6  39.7  15.4  12.2  46.6  31.3  22.3  23.1  17.4  46.8  32.9  62.9
  25.5  42.2  42.5  39.5  49.3  31.3  11.6  55.1  19.   34.6  50.8  63.2
  43.4  35.1  27.3  41.2  37.2  23.1  14.7  34.1  37.5  42.   25.6  53.3
  41.4  20.   47.4  24.7  38.5  49.   57.1  55.9  36.9  25.6  20.7  37.
  29.5  33.6  47.3  23.9  18.3  35.7  67.7  40.6  43.5  31.1  51.8  61.9
  56.3  63.3  48.1  18.6  20.5  20.9  27.3  26.6  34.   52.2  56.8  38.8
  44.   39.7  53.9  30.6  42.3  41.   16.1  39.4  73.6  44.5  36.7  30.5
  15.9  42.3  34.1  26.5  24.4  31.7  40.8  23.8  28.6  58.   37.3  42.9
  28.1  46.6  57.8  30.5  39.6  17.7  40.6  27.3  51.6  39.7  28.9  70.1
  45.4  28.4  11.2  42.5  39.   30.7  39.6  22.3  47.   47.7  23.   12.8
  26.9  54.8  52.2  48.6  25.6  21.8  42.8  35.3  49.7  40.5  23.6  22.8
  40.3  63.3  58.1  24.7  27.   23.7  22.9  42.5  22.6  18.2 117.5  44.2
  40.9  12.8  19.2  41.2  45.5  48.5  27.7  32.2  37.5  22.   47.9  34.1
  37.9  45.9  25.7  51.   29.4  27.7  36.7  26.5  41.   29.3  51.8  33.1
  23.5  42.   55.   13.4  34.2  31.1  40.2  45.1  40.1  15.5  30.7  42.1
  52.5  53.   31.3  46.1  43.7  48.   45.1  69.7  43.2  48.1  41.1  59.6
  54.4  53.5  37.7  43.9  43.1  34.7  71.   47.3  29.3  24.8  47.7  38.9
  47.1  60.7  37.   47.   18.8  55.5  44.7  21.7  31.5  25.   34.2  55.
  21.5  46.6  51.   23.8  43.2  55.3  40.3  42.   62.2  36.3  36.5  48.2
  48.2  46.4  38.1  36.3  38.4  27.   30.6  30.9  23.   45.2  35.5  46.2
  49.3  41.6  31.9  48.   59.   18.3  24.5  33.4  27.7  43.5  22.1  24.7
  34.6  13.8  42.5  42.7  13.   61.5  58.1]
X_test:
 [[2.0128330e+03 1.0300000e+01 2.1144730e+02 1.0000000e+00 2.4974170e+01
  1.2152999e+02]
 [2.0133330e+03 2.4000000e+01 4.5276870e+03 0.0000000e+00 2.4947410e+01
  1.2149628e+02]
 [2.0133330e+03 3.4500000e+01 3.2494190e+02 6.0000000e+00 2.4978140e+01
  1.2154170e+02]
 [2.0133330e+03 2.5600000e+01 4.5196900e+03 0.0000000e+00 2.4948260e+01
  1.2149587e+02]
 [2.0135000e+03 1.4400000e+01 1.6998030e+02 1.0000000e+00 2.4973690e+01
  1.2152979e+02]
 [2.0130830e+03 3.6600000e+01 4.8881930e+02 8.0000000e+00 2.4970150e+01
  1.2154494e+02]
```

```
[2.0132500e+03 3.5800000e+01 1.7073110e+02 7.0000000e+00 2.4967190e+01
 1.2154269e+02]
[2.0130830e+03 3.4800000e+01 4.0521340e+02 1.0000000e+00 2.4973490e+01
 1.2153372e+02]
[2.0134170e+03 1.0500000e+01 2.7917260e+02 7.0000000e+00 2.4975280e+01
 1.2154541e+02]
[2.0134170e+03 1.7200000e+01 3.9056840e+02 5.0000000e+00 2.4979370e+01
 1.2154245e+02]
[2.0132500e+03 1.3300000e+01 2.5063100e+02 7.0000000e+00 2.4966060e+01
 1.2154297e+02]
[2.0130830e+03 1.3300000e+01 3.3605320e+02 5.0000000e+00 2.4957760e+01
 1.2153438e+02]
[2.0128330e+03 1.2700000e+01 1.8748230e+02 1.0000000e+00 2.4973880e+01
 1.2152981e+02]
[2.0130830e+03 3.4000000e+01 1.5760520e+02 7.0000000e+00 2.4966280e+01
 1.2154196e+02]
[2.0128330e+03 4.6000000e+00 2.5966070e+02 6.0000000e+00 2.4975850e+01
 1.2154516e+02]
[2.0135830e+03 3.5900000e+01 6.1640040e+02 3.0000000e+00 2.4977230e+01
 1.2153767e+02]
[2.0131670e+03 3.2600000e+01 4.9365700e+02 7.0000000e+00 2.4969680e+01
 1.2154522e+02]
[2.0128330e+03 0.0000000e+00 1.8542960e+02 0.0000000e+00 2.4971100e+01
 1.2153170e+02]
[2.0129170e+03 1.3000000e+01 4.9223130e+02 5.0000000e+00 2.4965150e+01
 1.2153737e+02]
[2.0129170e+03 3.4800000e+01 1.9003920e+02 8.0000000e+00 2.4977070e+01
 1.2154312e+02]
[2.0126670e+03 5.7000000e+00 9.0456060e+01 9.0000000e+00 2.4974330e+01
 1.2154310e+02]
[2.0131670e+03 1.2000000e+01 1.3601390e+03 1.0000000e+00 2.4952040e+01
 1.2154842e+02]
[2.0129170e+03 3.0000000e+01 1.0133410e+03 5.0000000e+00 2.4990060e+01
 1.2153460e+02]
[2.0130830e+03 1.6100000e+01 2.8932480e+02 5.0000000e+00 2.4982030e+01
 1.2154348e+02]
[2.0129170e+03 2.1000000e+00 4.5124380e+02 5.0000000e+00 2.4975630e+01
 1.2154694e+02]
[2.0129170e+03 1.5000000e+01 3.8328050e+02 7.0000000e+00 2.4967350e+01
 1.2154464e+02]
[2.0134170e+03 1.8200000e+01 4.5164190e+02 8.0000000e+00 2.4969450e+01
 1.2154490e+02]
[2.0126670e+03 3.0400000e+01 1.7355950e+03 2.0000000e+00 2.4964640e+01
 1.2151623e+02]
[2.0132500e+03 5.4000000e+00 3.9056840e+02 5.0000000e+00 2.4979370e+01
 1.2154245e+02]
[2.0128330e+03 1.0300000e+01 3.0798900e+03 0.0000000e+00 2.4954600e+01
 1.2156627e+02]
[2.0135000e+03 4.1000000e+00 3.1289630e+02 5.0000000e+00 2.4955910e+01
 1.2153956e+02]
[2.0135000e+03 1.8100000e+01 8.3772330e+02 0.0000000e+00 2.4963340e+01
 1.2154767e+02]
[2.0128330e+03 0.0000000e+00 2.7401440e+02 1.0000000e+00 2.4974800e+01
 1.2153059e+02]
[2.0135830e+03 3.5700000e+01 5.7920830e+02 2.0000000e+00 2.4982400e+01
 1.2154619e+02]
```

```
[2.0129170e+03 1.7700000e+01 4.5164190e+02 8.0000000e+00 2.4969450e+01
 1.2154490e+02]
[2.0134170e+03 1.2200000e+01 1.3601390e+03 1.0000000e+00 2.4952040e+01
 1.2154842e+02]
[2.0131670e+03 3.3200000e+01 1.2172620e+02 1.0000000e+01 2.4981780e+01
 1.2154059e+02]
[2.0133330e+03 5.1000000e+00 1.5598270e+03 3.0000000e+00 2.4972130e+01
 1.2151627e+02]
[2.0129170e+03 6.8000000e+00 3.7955750e+02 1.0000000e+01 2.4983430e+01
 1.2153762e+02]
[2.0128330e+03 1.8000000e+01 6.3061530e+03 1.0000000e+00 2.4957430e+01
 1.2147516e+02]
[2.0134170e+03 0.0000000e+00 3.3896790e+02 9.0000000e+00 2.4968530e+01
 1.2154413e+02]
[2.0129170e+03 3.8200000e+01 5.5243710e+02 2.0000000e+00 2.4975980e+01
 1.2153381e+02]
[2.0128330e+03 1.5900000e+01 1.4977130e+03 3.0000000e+00 2.4970030e+01
 1.2151696e+02]
[2.0135000e+03 3.2300000e+01 1.0994550e+02 1.0000000e+01 2.4981820e+01
 1.2154086e+02]
[2.0133330e+03 1.6700000e+01 4.0820150e+03 0.0000000e+00 2.4941550e+01
 1.2150381e+02]
[2.0132500e+03 6.5000000e+00 3.7617090e+02 6.0000000e+00 2.4954180e+01
 1.2153713e+02]
[2.0127500e+03 2.9300000e+01 5.2977710e+02 8.0000000e+00 2.4981020e+01
 1.2153655e+02]
[2.0127500e+03 1.6100000e+01 4.0665870e+03 0.0000000e+00 2.4942970e+01
 1.2150342e+02]
[2.0134170e+03 1.1000000e+01 1.9312070e+03 2.0000000e+00 2.4963650e+01
 1.2151471e+02]
[2.0135000e+03 4.1000000e+00 5.6474250e+01 7.0000000e+00 2.4957440e+01
 1.2153711e+02]
[2.0135000e+03 6.5000000e+00 9.0456060e+01 9.0000000e+00 2.4974330e+01
 1.2154310e+02]
[2.0126670e+03 3.4500000e+01 6.2347310e+02 7.0000000e+00 2.4979330e+01
 1.2153642e+02]
[2.0129170e+03 1.9500000e+01 3.0659470e+02 9.0000000e+00 2.4980340e+01
 1.2153951e+02]
[2.0132500e+03 3.0300000e+01 1.2647300e+03 0.0000000e+00 2.4948830e+01
 1.2152954e+02]
[2.0130830e+03 1.1900000e+01 1.3601390e+03 1.0000000e+00 2.4952040e+01
 1.2154842e+02]
[2.0128330e+03 3.4900000e+01 5.6703490e+02 4.0000000e+00 2.4970030e+01
 1.2154580e+02]
[2.0132500e+03 3.8000000e+00 3.8386240e+02 5.0000000e+00 2.4980850e+01
 1.2154391e+02]
[2.0131670e+03 2.6600000e+01 4.8275810e+02 5.0000000e+00 2.4974330e+01
 1.2153863e+02]
[2.0127500e+03 1.2900000e+01 2.5063100e+02 7.0000000e+00 2.4966060e+01
 1.2154297e+02]
[2.0130830e+03 1.7700000e+01 8.3772330e+02 0.0000000e+00 2.4963340e+01
 1.2154767e+02]
[2.0135000e+03 3.5300000e+01 6.1657350e+02 8.0000000e+00 2.4979450e+01
 1.2153642e+02]
[2.0134170e+03 4.0100000e+01 1.2374290e+02 8.0000000e+00 2.4976350e+01
 1.2154329e+02]
```

```
 [2.0130830e+03 1.4000000e+01 4.3885130e+02 1.0000000e+00 2.4974930e+01
  1.2152730e+02]
 [2.0129170e+03 9.9000000e+00 2.7917260e+02 7.0000000e+00 2.4975280e+01
  1.2154541e+02]
 [2.0126670e+03 3.3400000e+01 1.8696860e+02 6.0000000e+00 2.4966040e+01
  1.2154211e+02]
 [2.0131670e+03 2.0900000e+01 2.1851280e+03 3.0000000e+00 2.4963220e+01
  1.2151237e+02]
 [2.0126670e+03 2.9400000e+01 4.5103590e+03 1.0000000e+00 2.4949250e+01
  1.2149542e+02]
 [2.0135830e+03 3.2100000e+01 1.4385790e+03 3.0000000e+00 2.4974190e+01
  1.2151750e+02]
 [2.0126670e+03 3.1000000e+00 3.8386240e+02 5.0000000e+00 2.4980850e+01
  1.2154391e+02]
 [2.0130830e+03 3.6600000e+01 4.8881930e+02 8.0000000e+00 2.4970150e+01
  1.2154494e+02]
 [2.0129170e+03 1.8900000e+01 1.0092350e+03 0.0000000e+00 2.4963570e+01
  1.2154951e+02]
 [2.0128330e+03 5.0000000e+00 3.9056840e+02 5.0000000e+00 2.4979370e+01
  1.2154245e+02]
 [2.0133330e+03 3.7200000e+01 1.8651010e+02 9.0000000e+00 2.4977030e+01
  1.2154265e+02]
 [2.0128330e+03 1.5000000e+01 1.8283190e+03 2.0000000e+00 2.4964640e+01
  1.2151531e+02]
 [2.0132500e+03 3.1500000e+01 4.1494760e+02 4.0000000e+00 2.4981990e+01
  1.2154464e+02]
 [2.0135000e+03 4.0000000e+00 2.1473760e+03 3.0000000e+00 2.4962990e+01
  1.2151284e+02]
 [2.0126670e+03 7.1000000e+00 2.1750300e+03 3.0000000e+00 2.4963050e+01
  1.2151254e+02]
 [2.0128330e+03 1.3900000e+01 4.5737790e+03 0.0000000e+00 2.4948670e+01
  1.2149507e+02]
 [2.0134170e+03 1.6400000e+01 2.8932480e+02 5.0000000e+00 2.4982030e+01
  1.2154348e+02]
 [2.0135000e+03 1.7500000e+01 9.6474960e+02 4.0000000e+00 2.4988720e+01
  1.2153411e+02]
 [2.0134170e+03 2.0300000e+01 2.8760250e+02 6.0000000e+00 2.4980420e+01
  1.2154228e+02]
 [2.0129170e+03 1.4700000e+01 1.3601390e+03 1.0000000e+00 2.4952040e+01
  1.2154842e+02]
 [2.0134170e+03 3.0400000e+01 4.6422300e+02 6.0000000e+00 2.4979640e+01
  1.2153805e+02]]
y_test:
 [45.3 14.4 46.  15.6 50.2 38.1 48.5 41.4 51.6 40.1 42.  42.4 28.5 39.1
 53.7 36.8 40.6 55.2 39.3 44.3 53.5 25.3 22.8 51.7 45.5 34.4 31.6 25.9
 49.5 24.7 51.7 29.7 45.4 50.5 26.5 30.1 46.1 28.9 54.4 15.  44.9 29.8
 23.6 48.  16.7 40.9 40.2 12.9 21.3 62.1 63.9 40.3 42.2 19.1 28.4 28.5
 60.7 37.5 39.3 28.8 42.3 44.3 42.6 57.4 42.2 25.7 13.2 27.  56.2 38.3
 20.8 43.1 78.3 20.9 32.5 33.4 32.1 19.2 53.  38.2 46.7 24.6 36.2]
Jumlah data train: 331 | Jumlah data test: 83
```

4. Memanggil LinearRegression()

In [ ]:
```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
```

```
regressor.fit(X_train, y_train)
```

Out[ ]:

▾ LinearRegression

▸ Parameters

▸ Fitted attributes

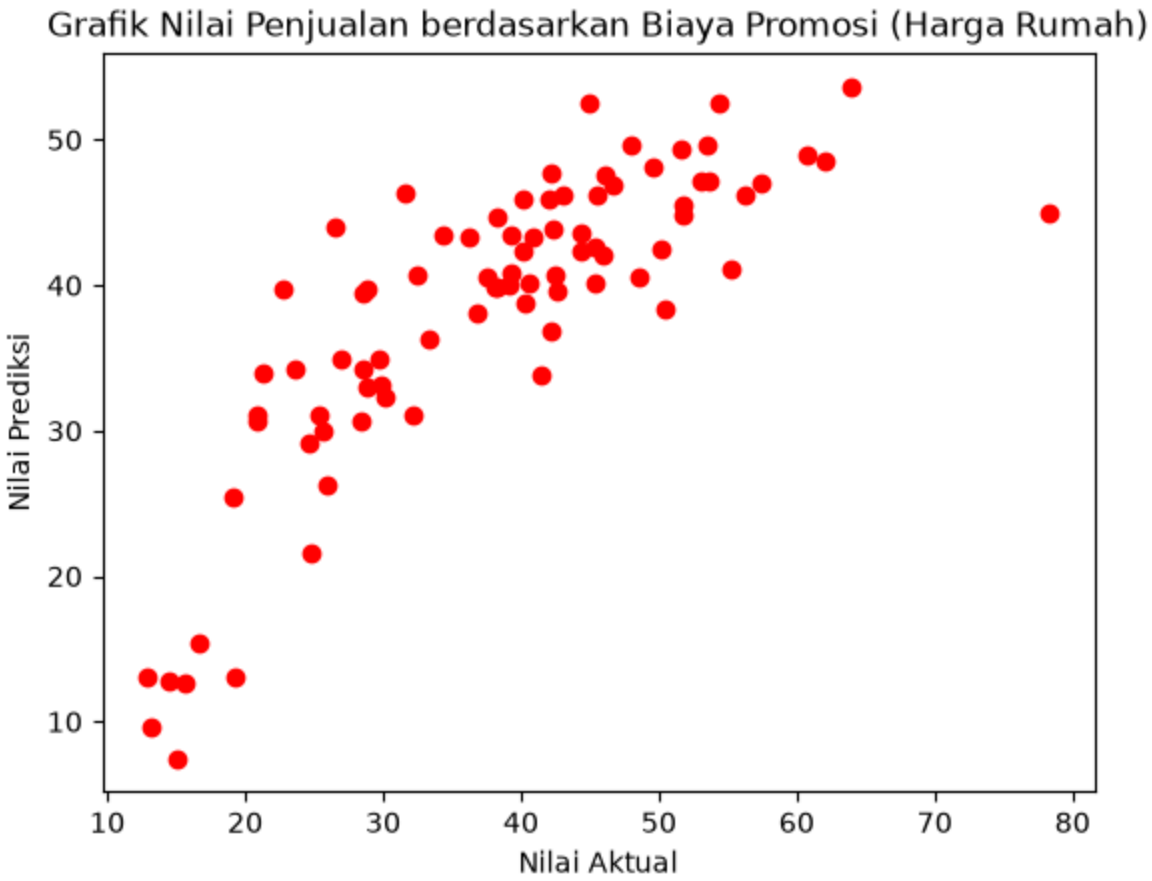
5. Menampilkan hasil predict dari data test

In [ ]:
```
y_pred = regressor.predict(X_test)
y_pred
```

Out[ ]:
```
array([40.09548022, 12.83819172, 42.00966923, 12.64733002, 42.44343803,
       39.83198984, 40.55595745, 33.86023762, 49.37328265, 45.87031976,
       45.86310332, 40.64517161, 39.50676203, 40.04955455, 47.11860745,
       38.13995857, 40.08417262, 41.12540438, 40.83964717, 42.37860913,
       49.70003677, 31.08231285, 39.67622569, 45.52764709, 46.18872415,
       43.43143246, 46.35880414, 26.20749681, 48.13688227, 21.53997799,
       44.8104349 , 34.94131598, 42.6644548 , 38.37379621, 43.98018394,
       32.28477958, 47.62406077, 39.77407301, 52.55712201,  7.46443039,
       52.54110472, 33.1285138 , 34.22107416, 49.59242876, 15.37567885,
       43.36042455, 42.40643117, 12.9983919 , 33.99155573, 48.50179814,
       53.67150141, 38.73047082, 47.73879852, 25.42163729, 30.68691477,
       34.27187582, 48.91161249, 40.61983209, 43.45816985, 32.95307348,
       43.85237072, 43.62659423, 39.55165632, 47.0209757 , 36.84017788,
       30.00963218,  9.66438363, 34.92190591, 46.16892784, 39.83198984,
       31.0777362 , 46.14863976, 44.93807805, 30.68551351, 40.64359986,
       36.24180038, 31.13625745, 13.07226688, 47.12551189, 44.71774583,
       46.8446933 , 29.11676154, 43.25162638])
```

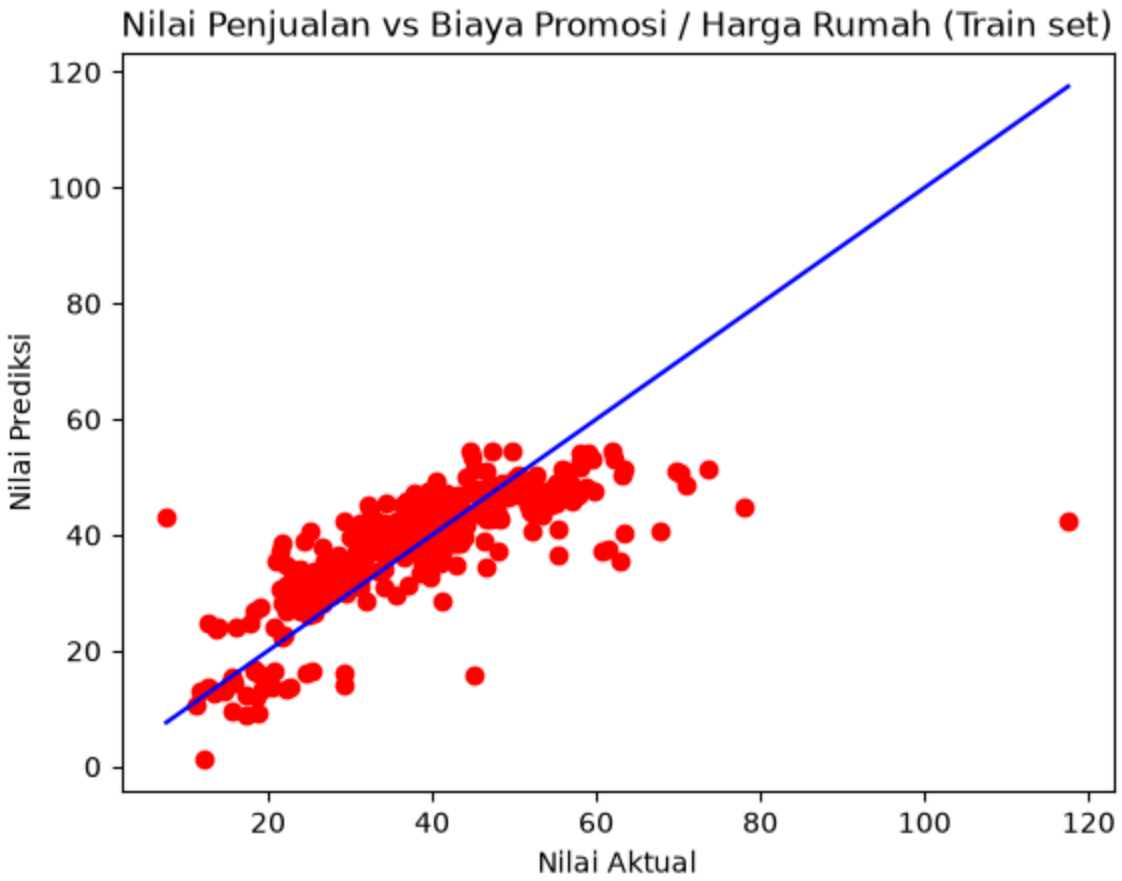
6. Menampilkan Grafik Nilai Penjualan berdasarkan Biaya Promosi

In [ ]:
```
plt.scatter(y_test, y_pred, color='red')
plt.title('Grafik Nilai Penjualan berdasarkan Biaya Promosi (Harga Rumah)')
plt.xlabel('Nilai Aktual')
plt.ylabel('Nilai Prediksi')
plt.show()
```

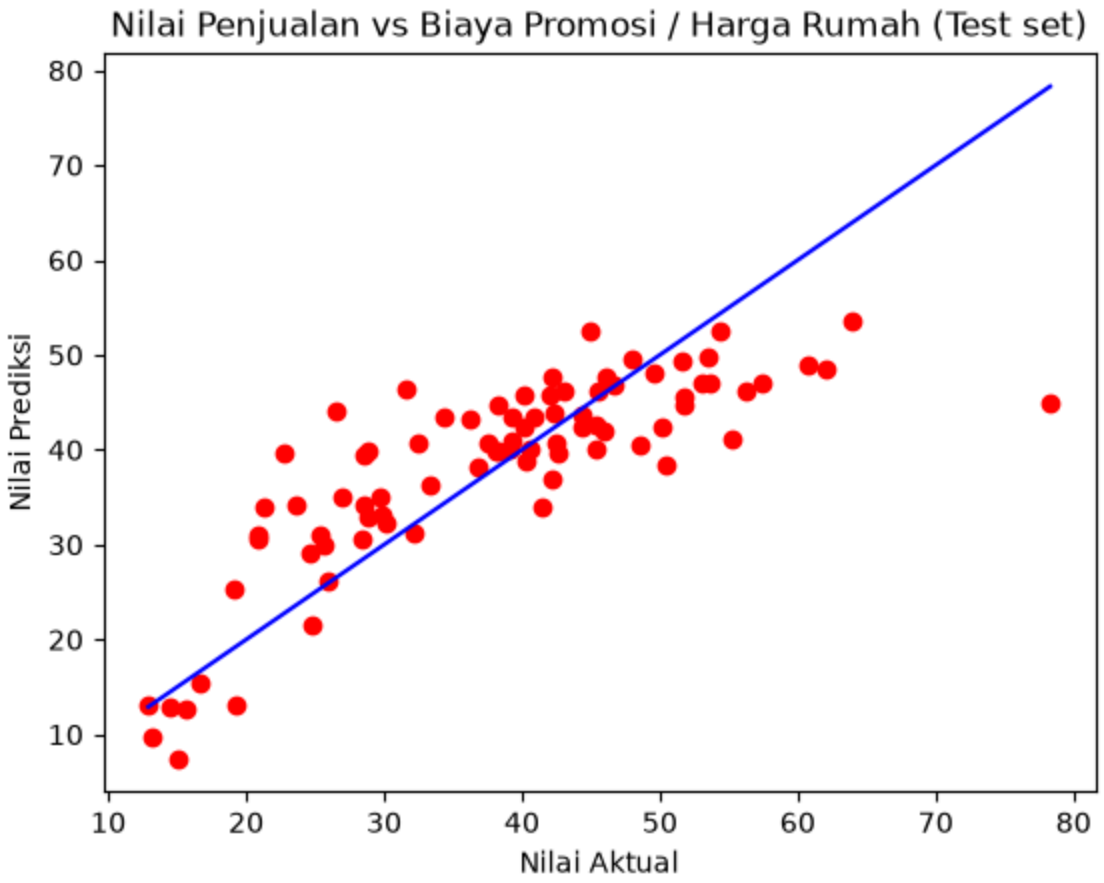


7. Menampilkan Garis hasil prediksi dari machine learning pada Biaya Promosi terhadap penjualan (train set dan test set)

In [ ]:
```
plt.scatter(y_train, regressor.predict(X_train), color='red')
plt.plot([y_train.min(), y_train.max()], [y_train.min(), y_train.max()], color='blu
plt.title('Nilai Penjualan vs Biaya Promosi / Harga Rumah (Train set)')
plt.xlabel('Nilai Aktual')
plt.ylabel('Nilai Prediksi')
plt.show()
```



In [ ]:
```
plt.scatter(y_test, y_pred, color='red')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='blue')
plt.title('Nilai Penjualan vs Biaya Promosi / Harga Rumah (Test set)')
plt.xlabel('Nilai Aktual')
plt.ylabel('Nilai Prediksi')
plt.show()
```



2. Hitunglah Intercept, Slope, dan R-Square

In [ ]:
```
a = regressor.intercept_
b = regressor.coef_

print("a. Estimated model intercept, a:", a)
print("b. Estimated model slope, b:", b)
for i, col in enumerate(df.columns[1:-1]):
    print(f"   Slope b{i+1} ({col}): {b[i]}")
```

```
a. Estimated model intercept, a: -14866.72182856487
b. Estimated model slope, b: [ 5.02037302e+00 -2.63132611e-01 -4.46345490e-03  1.092
58054e+00
  2.25488903e+02 -6.80648392e+00]
   Slope b1 (X1 transaction date): 5.020373024245612
   Slope b2 (X2 house age): -0.26313261092924306
   Slope b3 (X3 distance to the nearest MRT station): -0.004463454895057006
   Slope b4 (X4 number of convenience stores): 1.0925805383193314
   Slope b5 (X5 latitude): 225.48890340681132
   Slope b6 (X6 longitude): -6.806483919607358
```

In [ ]:
```
from sklearn.metrics import r2_score
score = r2_score(y_test, y_pred)
print("c. R-Square:", score)
```

```
c. R-Square: 0.6573242742218104
```

Soal No 3

1. Membaca file konsumen.csv

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('konsumen.csv', sep=';')
print("Dimensi dataset konsumen (baris, kolom):", df.shape)
```

```
Dimensi dataset konsumen (baris, kolom): (20, 3)
```

2. Menampilkan sebanyak 7 row data teratas

In [2]:
```
df.head(7)
```

Out[2]:

	No	Gaji	Pengeluaran
0	1	2500	1750
1	2	3800	4200
2	3	3900	3800
3	4	4350	5500
4	5	4400	3200
5	6	5500	5450
6	7	5600	5950

3. Menghapus kolom No

In [3]:
```
df = df.drop(columns=['No'])
print("Kolom 'No' berhasil dihapus. Dimensi data sekarang:", df.shape)
```

```
Kolom 'No' berhasil dihapus. Dimensi data sekarang: (20, 2)
```

4. Menampilkan 9 row data teratas

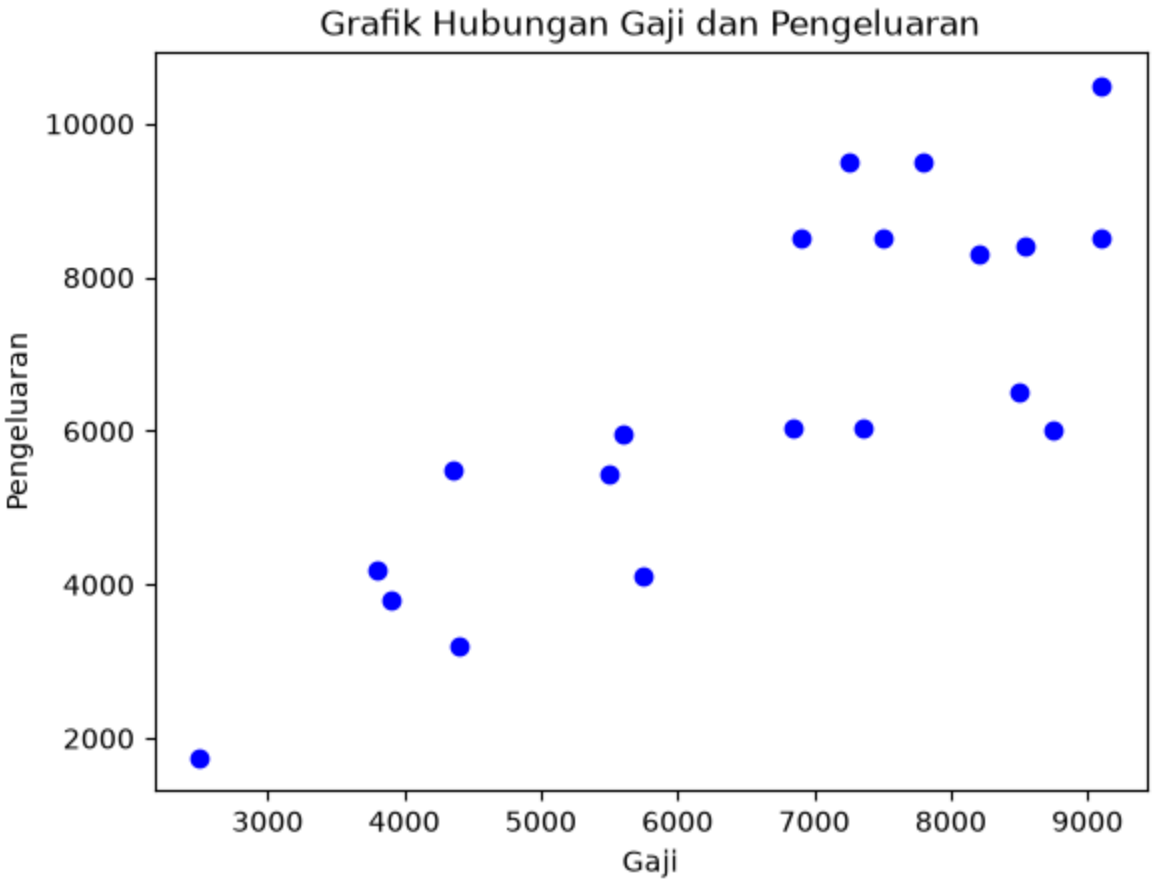
In [4]:
```
df.head(9)
```

Out[4]:

	Gaji	Pengeluaran
0	2500	1750
1	3800	4200
2	3900	3800
3	4350	5500
4	4400	3200
5	5500	5450
6	5600	5950
7	5750	4100
8	6850	6050

5. Menampilkan grafik hubungan gaji dan pengeluaran

In [6]:
```
plt.scatter(df['Gaji'], df['Pengeluaran'], color='blue')
plt.title('Grafik Hubungan Gaji dan Pengeluaran')
plt.xlabel('Gaji')
plt.ylabel('Pengeluaran')
plt.show()
```



6. Gunakan 2 klaster

In [7]:
```
from sklearn.cluster import KMeans

X = df[['Gaji', 'Pengeluaran']].values

kmeans = KMeans(n_clusters=2, random_state=0)
kmeans.fit(X)

print("Hasil label klaster tiap konsumen:\n", kmeans.labels_)
```

```
Hasil label klaster tiap konsumen:
 [1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0]
```

7. Menampilkan nilai pusat klaster

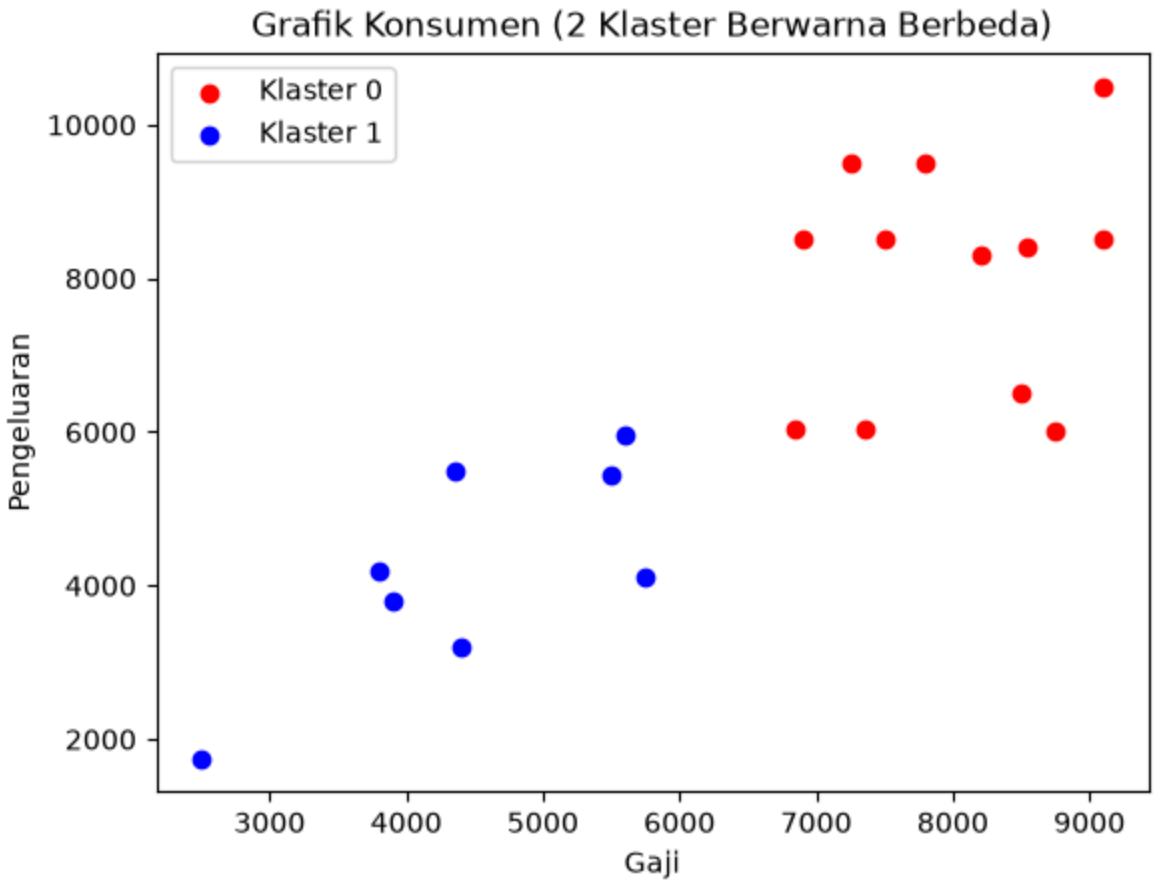
In [8]:
```
pusat_klaster = kmeans.cluster_centers_
print("Nilai pusat klaster (Centroid):\n", pusat_klaster)

for i, center in enumerate(pusat_klaster):
    print(f"Klaster {i}: Gaji rata-rata = {center[0]:.2f}, Pengeluaran rata-rata =
```

```
Nilai pusat klaster (Centroid):
 [[7987.5  8025.  ]
 [4475.   4243.75]]
Klaster 0: Gaji rata-rata = 7987.50, Pengeluaran rata-rata = 8025.00
Klaster 1: Gaji rata-rata = 4475.00, Pengeluaran rata-rata = 4243.75
```

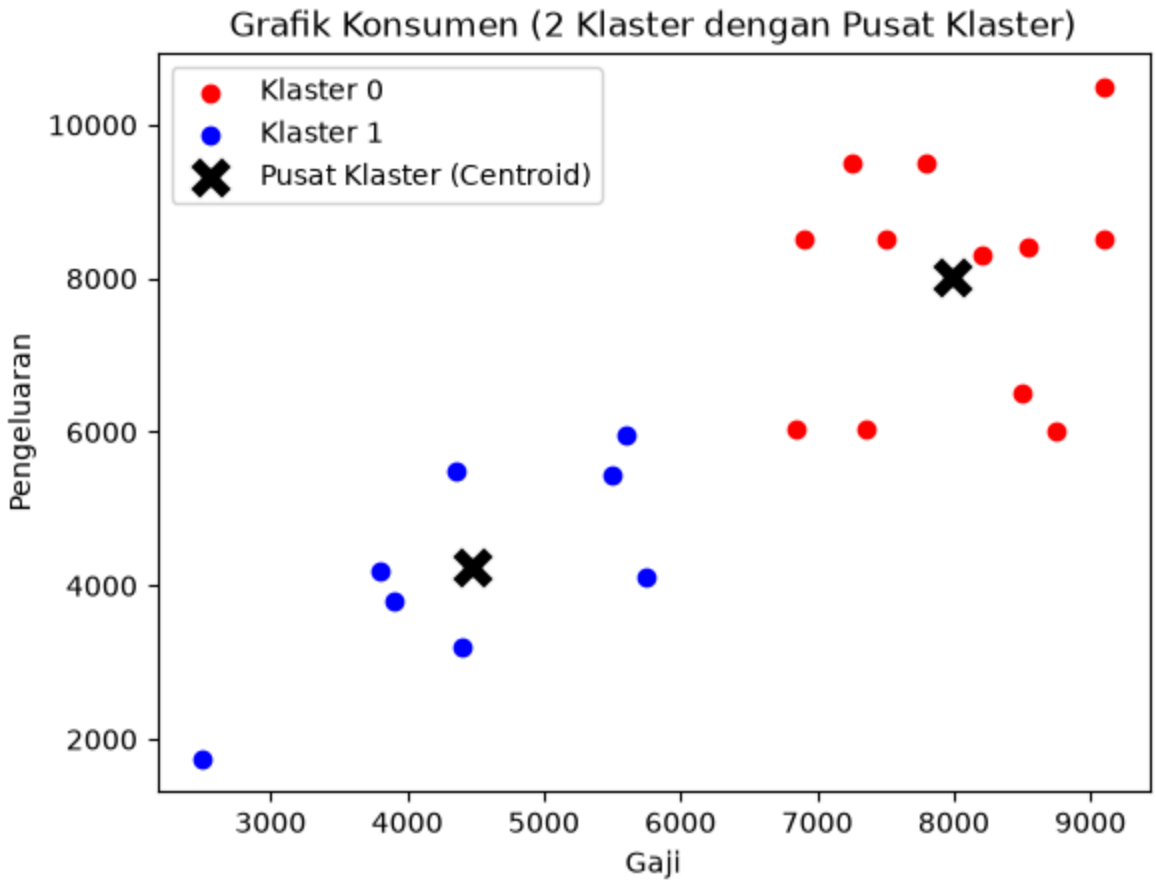
8. Menampilkan Grafik Konsumen yang memiliki 2 warna berbeda

In [9]:
```
plt.scatter(X[kmeans.labels_ == 0, 0], X[kmeans.labels_ == 0, 1], color='red', labe
plt.scatter(X[kmeans.labels_ == 1, 0], X[kmeans.labels_ == 1, 1], color='blue', lab
plt.title('Grafik Konsumen (2 Klaster Berwarna Berbeda)')
plt.xlabel('Gaji')
plt.ylabel('Pengeluaran')
plt.legend()
plt.show()
```



9. Menampilkan Grafik Konsumen yang memiliki 2 warna berbeda dengan pusat klaster

In [10]:
```
plt.scatter(X[kmeans.labels_ == 0, 0], X[kmeans.labels_ == 0, 1], color='red', labe
plt.scatter(X[kmeans.labels_ == 1, 0], X[kmeans.labels_ == 1, 1], color='blue', lab
plt.scatter(pusat_klaster[:, 0], pusat_klaster[:, 1], s=250, color='black', marker=
plt.title('Grafik Konsumen (2 Klaster dengan Pusat Klaster)')
plt.xlabel('Gaji')
plt.ylabel('Pengeluaran')
plt.legend()
plt.show()
```


Grafik Konsumen (2 Klaster dengan Pusat Klaster)
Klaster 0
Klaster 1
Pusat Klaster (Centroid)
Pengeluaran
Gaji
10000
8000
6000
4000
2000
3000
4000
5000
6000
7000
8000
9000